DAY 1

used : programiz c++ compliler
Real time code

```cpp
#include <iostream>
using namespace std;

// // User-defined function
// int add(int a, int b) {
//     return a + b;
// }
int sum(int num1, int num2) {
       return num1+num2;
   }

int main() {
  // const int x=20;
  //  cin>>x;
  //  cout<<x;

   int summ=sum(10,20);
   cout<<summ<<endl;
   // 1. Basic syntax and output
   cout << "Hello World!" <<endl;


   // 2. Data types
   // int age = 20;            // Integer
   char grade = 'A';        // Character
   float price = 10.5f;     // Single-precision decimal
   double pi = 3.14159;      // Double-precision decimal
   bool isStudent = true;    // Boolea
  // cout<<age<<endl;
   // 3. Variables and constants
   int x = 10;
   x=50;
   int y=10;
   cout<<x;
   const int MAX = 100;


   int result=add(x,y);
   int res=add(10,20);
   cout<<result;
   cout<<res;

   int z=max(x,y);
   cout<<z<<endl;
```

```cpp
// doubt-pow function


// int class=20;
// cout<<class;
// Constant: cannot be reassigned

// cout << "\nData Types and Variables:" << endl;
// cout << age << " " << grade << " " << price << endl;
// cout << pi << " " << isStudent << endl;
// cout << "x = " << x << ", MAX = " << MAX << endl;

// // 4. Keywords
// // int, char, if, else, return, const, void are C++ keywords.
// // Keywords have predefined meanings and cannot be used as variable names.

// // 5. Typecasting
// double num = 10.75;

// // Implicit typecasting
// int implicitValue = 5.9;  // double converted to int

// // Explicit typecasting
// int explicitValue = static_cast<int>(num);

// cout << "\nTypecasting:" << endl;
// cout << "Implicit: " << implicitValue << endl;
// cout << "Explicit: " << explicitValue << endl;

// 6. Pointers
int value = 20;
int* ptr = &value;       // Pointer stores the address of value

cout << "\nPointers:" << endl;
cout << "Value = " << value << endl;
cout << "Address = " << &value << endl;
cout << "Pointer stores = " << ptr << endl;
cout << "Value through pointer = " << *ptr << endl;

// // * is used to declare/dereference a pointer.
// // & gives the address of a variable.

// 7. Reference variables
int original = 10;
int& ref = original;      // ref is an alias for original
ref = 50;

cout << "\nReferences:" << endl;
cout << "Original = " << original << endl;
cout << "Reference = " << ref << endl;
```

```
// // 8. Functions
// int result = add(10, 20);

// cout << "\nFunctions:" << endl;
// cout << "Sum = " << result << endl;

// 9. Operators
int a = 10, b = 3,c=5;
// float c=float(a/b;)

cout << "\nArithmetic Operators:" << endl;
cout << "a + b = " << a + b << endl;
cout << "a - b = " << a - b << endl;
cout << "a * b = " << a * b << endl;
cout << "a / b = " << a / b << endl;
cout << "a % b = " << a % b << endl;
//10/3=3,remainder=1

cout << "\nRelational Operators:" << endl;
cout << (a > b) << endl;
cout << (a == b) << endl;

cout << "\nLogical Operators:" << endl;
cout << (a > 5 && b < 5) << endl;
cout << (a > 15 || b < 5) << endl;
cout << !(a == b) << endl;

cout << "\nAssignment Operators:" << endl;
a += 5;//a=a+5
a++;//a=a+1
cout << "a after += 6: " << a << endl;

cout << "\nBitwise Operators:" << endl;
cout << (5 & 3) << endl;  // Bitwise AND//5-101 3-011--001=1
cout << (5 | 3) << endl;  // Bitwise OR//111=7
cout << (5 ^ 3) << endl;  // Bitwise XOR

// 10. Conditional statements
int marks = 75;

cout << "\nConditional Statements:" << endl;

if (marks >= 90) {
    cout << "Grade A+" << endl;
    cout<<"excellent";
}
    else if (marks>=70) cout<<"b grade"<<endl;
else
    cout << "Needs improvement" << endl;
```

```
// Switch statement
int choice = 2;

switch (choice) {
    case 1:
        cout << "Option 1" << endl;
        break;
    case 2:
        cout << "Option 2" << endl;
        break;
    default:
        cout << "Invalid option" << endl;
}

//  int age;
// cout<<age;
 // cout<<"Enter your age(18+) :"<<endl;
 // cin>>age;
 // cout<<"your age is : " <<name;

 // // 11. Loops

 // For loop
 cout << "\nFor Loop: ";
 for (int i = 1; i <= 5; i++) {
    cout << i << " ";//1 2 3 4 5
 }

 // While loop
 cout << "\nWhile Loop: ";
 int i = 1;
 while (i <= 3) {
    cout << i << " ";//1 2 3
    i++;
 }

 // Do-while loop
 cout << "\nDo-While Loop: ";
 int j = 1;
 do {
    cout << j << " ";
    j++;
 } while (j <= 3);

 // // 12. Jump statements

 // break: terminates the loop
 cout << "\nBreak: ";
 for (int k = 1; k <= 5; k++) {
    if (k == 3)
```

```
            break;
        cout << k << " ";//1 2
    }

    // continue: skips the current iteration
    cout << "\nContinue: ";
    for (int k = 1; k <= 5; k++) {
        if (k == 3)
            continue;
        cout << k << " "<<endl;//1 2 4 5
    }
   double d=3.14;
    int e=(int)d;
    cout<<e;
    // // return: terminates the function
    // cout << "\nProgram completed." << endl;
   return 0;
}
```

NOTES

Day 1

15 September 2024 20:40

⇒ Hello World!

- Header files
  - System defined → eg:- iostream, vector, string
  - User defined → #include<filename>
- namespace
- Main function → entry point of the code

⇒ Data types



⇒ Keywords → reserved identifiers, specific C++ language features

↓

names of variables, arrays, functions, classes etc.

⇒ Functions → any code that is repeated is usually placed in a function

⇒ Operators

- Logical
- Arithmetic
- Assignment
- Bitwise

⇒ Variables and Constants

- value changes. (eg:- 1,2,3,4...)
- Memory allocated to variables depends on its data type.

The general rules for naming variables are:

- Names can contain letters, digits and underscores
- Names must begin with a letter or an underscore (_)
- Names are case-sensitive (myVar and myvar are different variables)
- Names cannot contain whitespaces or special characters like !, #, %, etc.
- Reserved words (like C++ keywords, such

variables depends on its data type.

int → 4 bytes, char → 1 byte etc.

(myVar and myvar are different variables)
- Names cannot contain whitespaces or special characters like !, #, %, etc.
- Reserved words (like C++ keywords, such as int) cannot be used as names

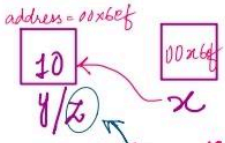
⇒ Typecasting → converting one data type to another.

implicit ↔ explicit

→ Pointers → variables pointing to the address of another variable

* → value of (address) operator
& → address of (variable) operator

```
int *x;  → pointer variable
int y = 10;
x = &y;
int &z = y;
```



z is a reference variable → just an alias for the referred variable.

Note :-

https://www.geeksforgeeks.org/ascii-table/ → ASCII values

https://www.geeksforgeeks.org/operator-precedence-and-associativity-in-cpp/ → operator precedence & associativity

LEETCODE QUES

